Frank Zhao

College Station, TX || yuyangzhao037@gmail.com || frankzhao77.github.io

SUMMARY

Computer Science senior at Texas A&M University graduating May 2027, with internship experience developing ERP workflows and projects in full-stack development and AI-powered applications. Strongest in Java, with hands-on experience in Python, Go, React, and Spring Boot. Seeking a **Summer 2027 Software Engineering Internship** or a **full-time role upon graduation**.

EDUCATION

Texas A&M University College Station, TX, US
Bachelor of Science, Computer Science, GPA 3.7/4.0 May 2023 – May 2027 (expected)

TECHNICAL SKILLS

Java/Kotlin	Python/Go	JavaScript/TypeScript	C/C++	Ruby/C#	R/MATLAB
HTML/CSS	React/Redux	AngularJS/jQuery	Android/Swift	Java Servlet	Linux
Spring/Node.js	MySQL/PostgreSQL	AWS Lambda/EC2/RDS	Google GCE/GAE	OpenAI API	RAG/FAISS
YOLO/CLIP	Data Modeling	Feature Engineering	Recommendation Systems		

WORK EXPERIENCE

Full-Stack Software Engineer Intern May 2026 – August 2026
SpectraCell Laboratories, Inc.

- Delivered five merged full-stack enhancements to streamline procurement, strengthen data integrity, and improve usability.
- Customized cloud ERP with forms, validations, reports, workflows, and integrations for procurement, inventory, and lab operations.
- Developed with **Python, JavaScript, Frappe/ERPNext, MariaDB, and Docker**, using Kanban, Git branches, testing, and code reviews.
- Added notes, creator tracking, warehouse defaults, and required UOM selection to Material Requests, cutting creation time by **30%**.
- Built a price-change approval workflow linking purchase orders, invoices, and receipts to centralize history and improve auditability.

PROJECT EXPERIENCES

Agent AI: Full-Stack RAG & MCP-Based Document Q&A Agent

- Engineered a full-stack conversational AI system using **React + Ant Design**, integrating **voice I/O modules (react-speech-recognition, speak-tts)** to enable seamless bidirectional spoken interaction, reducing user query time by **40%**.
- Implemented a **Retrieval-Augmented Generation (RAG)** pipeline with **LangChain and GPT-5**, achieving **92%** semantic retrieval accuracy and generating context-aware responses across multi-document datasets.
- Designed a hybrid retrieval strategy that combined local **vector search (FAISS)** with web-augmented retrieval via **Model Context Protocol (MCP)** integrating SerpAPI for dynamic real-time knowledge injection.
- Built and deployed a **Node.js + Express backend** to handle document ingestion, query orchestration, and **RAG inference**, supporting 200+ concurrent queries with sub-second latency under production load testing.

Favify: A Kotlin-Based Spotify Favorites App with Optimized Performance and Enhanced Interaction

- Designed a **Kotlin-based** Spotify favorite app utilizing **Android Jetpack Library** and **Hilt Dependency Injection**, which streamlined app performance and maintenance.
- Implemented BottomBar & App Navigation using **Jetpack Navigation** component, improving user navigation efficiency.
- Created a mock **RESTful API** using **json-server** and utilized **Retrofit** to manage API requests.
- Built the feed/album/favorite UI in **Jetpack Compose** following **MVVM** architecture, enhancing UI responsiveness and user interaction.
- Enabled local cache ability for favorite features using **Room Database**.
- Integrated **Google ExoPlayer** for global music playback, improving playback stability and user satisfaction.
- Deployed backend services in **Docker** containers and orchestrated them using **Kubernetes**, ensuring efficient scaling and high availability.

PermitAI: AI-Assisted Municipal Permit Intake and Review Portal

- Developed a full-stack municipal permit portal using **React 19, TypeScript, Node.js, Express, and PostgreSQL**, supporting resident applications, document intake, staff review, permit issuance, and status tracking.
- Integrated the **OpenAI Responses API** with a predefined schema to extract **23 structured fields** from uploaded permit documents and calculate document completeness scores.
- Implemented **deterministic TypeScript validation** to compare **13 extracted fields** against application, resident, and property records using address normalization, name matching, date normalization, and description similarity rules.
- Automated application routing by recording missing fields and conflicts, denying conflicting submissions, and sending valid or extraction-failed applications to a staff review queue.
- Built persistent audit records and queued email notifications with retry handling using **PostgreSQL and Nodemailer**; containerized the frontend, backend, and database with **Docker Compose** and validated functionality using **Vitest, React Testing Library, ESLint, and TypeScript checks**.